

Tema 03

Funciones en C

Modularización y reutilización de código

Informática Industrial y Comunicaciones

Objetivo del tema

1. Entender qué es una función en C.
2. Aprender su sintaxis y estructura.
3. Comprender cómo se pasan argumentos y se devuelve un resultado.
4. Aplicar funciones para dividir programas en partes más claras y reutilizables.

¿Qué es una función?

Las partes repetitivas de un programa se encapsulan en funciones para reutilizarlas y organizar mejor el código.



Desde fuera, una función se comporta como una **caja negra**: recibe entradas, procesa algo internamente y devuelve una salida.

Declaración de una función

El código de una función sólo depende de sí misma: recibe nombres generales para su identificador, parámetros y variables locales.

- Debe estar declarada antes de ser utilizada en el resto del programa.
- Su estructura general es:
`tipoRetorno nombreFuncion(tipo1 p1, tipo2 p2, ...).`
- Dentro de la función se usan solo sus parámetros y variables locales.
- `return` devuelve el resultado calculado.

Sintaxis básica

```
#include <stdio.h>

int mayor(int a, int b) {
    int resultado;
    if (a > b)
        resultado = a;
    else
        resultado = b;
    return resultado;
}

int main(void) {
    int x = 3;
    int y = 5;
    int m = mayor(x, y);
    printf("%d\n", m);
    return 0;
}
```

Tipo de retorno:

int: devuelve un entero

Nombre de la función

mayor

Parámetros de entrada

(int a, int b)

Cuerpo

{...}

Una función debe estar **declarada** antes de poder ser usada en el resto del código.

Tipos de retorno

El tipo de retorno indica el tipo de dato que la función produce.

Puede ser **cualquier tipo válido en C**, por ejemplo:

- **int**: devuelve un entero.
- **float** o **double**: devuelven valores reales.
- **char**: devuelve un carácter.
- **void**: la función no devuelve ningún valor.

La instrucción `return` se usa para enviar ese valor al punto donde la función fue llamada.

La función debe devolver exactamente un valor del tipo declarado, o no devolver nada si el tipo es `void`.

Paso de parámetros

En C, los parámetros se pasan por valor:

- Se copia el valor de la variable de la llamante en el parámetro de la función.
- La función trabaja con esa copia, no con la variable original.
- Si cambia el parámetro dentro de la función, la variable original no se modifica.

Idea clave: la función recibe una copia del valor, no la variable completa.

Ejemplo: paso por valor

SIN return

```
void incrementar(int x) {  
    x = x + 1;  
    printf("Dentro: %d\n", x);  
}  
  
int main(void) {  
    int n = 10;  
    incrementar(n);  
    printf("Fuera: %d\n", n);  
}
```

Salida: Dentro: 11 y Fuera:
10

CON return

```
int incrementar(int x) {  
    x = x + 1;  
    return x;  
}  
  
int main(void) {  
    int n = 10;  
    n = incrementar(n);  
    printf("Ahora: %d\n", n);  
}
```

Salida: Ahora: 11

En el primer caso la función modifica una copia local; en el segundo, el valor devuelto por return se asigna de nuevo a la variable de la llamante.

Funciones con retorno

Una función puede devolver un resultado que será usado en la función que la invoca.

- **return** termina la ejecución de la función.
- El valor devuelto debe coincidir con el tipo especificado.
- Esto hace posible construir programas más modulares.

Ejemplo de reutilización

Supongamos que queremos calcular $m! / (m-n)!$

- El cálculo de ambos factoriales es idéntico.
- Lo que cambia son los valores iniciales de entrada.
- Por eso conviene encapsularlo en una función reutilizable.
- La variable índice sirve como auxiliar temporal dentro de la función.

Ejemplo práctico

La definición de `factorial` aparece antes de su uso en `main`.

```
#include <stdio.h>

int factorial(int n) {
    int indice;
    int resultado = 1;

    for (indice = 2; indice <= n; indice++) {
        resultado *= indice;
    }
    return resultado;
}

int main(void) {
    int m = 5;
    int n = 2;

    int resultado = factorial(m) / (factorial(m-n));
    return 0;
}
```

Funcionamiento interno

Cuando una función es invocada:

- Se deja temporalmente la función llamante.
- Las variables locales de la llamada anterior se mantienen, pero no son accesibles dentro de la función invocada.
- Las variables locales y los parámetros de la función invocada existen solo mientras se ejecuta esa función.
- Los parámetros reciben su valor inicial en la invocación, pero son variables nuevas y no afectan a las de la función llamante.
- Al regresar, sólo se puede utilizar el valor devuelto.

Ámbito y visibilidad

Las variables pueden tener distintos ámbitos:

- **Variables locales:** existen dentro de la función.
- **Variables globales:** existen en todo el programa.

Las funciones ayudan a encapsular lógica y evitar repetir código.

Ventajas de usar funciones

- Reducen la duplicación de código.
- Facilitan la depuración.
- Mejoran la legibilidad.
- Permiten reutilizar rutinas en distintos programas.

Funciones y estructuras

- Las funciones pueden recibir y devolver valores de tipo estructura.
 - Al igual que con el resto de tipos, se realiza una **copia local** para los parámetros y se copia en la variable destino el resultado.
(En estructuras grandes, estas copias pueden ser muy lentas).
- Ejemplo:

```
struct complejo
{
    float re, im;
};

struct complejo ProdComplejos(struct complejo a, struct complejo b)
// Calcula y devuelve a*b
{
    struct complejo prod;
    prod.re = a.re * b.re - a.im * b.im;
    prod.im = a.re * b.im + a.im * b.re;
    return prod;
}

...
// En otra zona del programa (función llamante)
struct complejo uno_menos_j, uno_menos_j_cuadrado;
uno_menos_j.re = 1; uno_menos_j.im = -1;
uno_menos_j_cuadrado = ProdComplejos(uno_menos_j, uno_menos_j);
```

Ejercicios Propuestos

Realizar utilizando funciones (cada ejercicio utilizará funciones realizadas en los anteriores):

1. Calcular el máximo común divisor (MCD) de 2 enteros mediante el algoritmo de Euclides (https://es.wikipedia.org/wiki/Algoritmo_de_Euclides).

2. Calcular el mínimo común múltiplo (MCM) de 2 enteros: $\text{mcm}(a, b) = (a \cdot b) / \text{mcd}(a, b)$

3. Se divide una misma longitud utilizando 3 escalas (A cada 12 m, B cada 18 m, C cada 30 m):

¿En qué punto después del origen coincidirán por 1ª vez las marcas de las 3 escalas?

$$\text{mcm}(a, b, c) = \text{mcm}(\text{mcm}(a, b), c)$$

4. ¿Cuál es el área del círculo cuyo centro es la primera coincidencia de las escalas A y B (36), y que pasa por la primera coincidencia de las 3 escalas (90)?

Coincidencia periódica de marcas (Escala de medición)

Escala A (12m):	0 . . . 12 . . . 24 . . . 36 . . . 48 . . . 60 . . . 72 . . . 84 . . . 90
Escala B (18m):	0 18 36 54 72 90
Escala C (30m):	0 30 60 90

Resumen

1. Una función agrupa una tarea concreta.
2. Puede recibir parámetros de entrada.
3. Puede devolver un resultado con `return`.
4. Mejora la estructura y reutilización del programa.